CompactifAI Aim: Implementation and Reproduction Report

Target paper: `CompactifAI Extreme Compression of Large Language Models using Quantum-Inspired Tensor Networks.pdf` **Related evidence:** the papers in `../research_paper_reference/` and the synthesis in `../research_paper_reference/research_paper_analysis_report.md` **Prepared:** 2026-11-06

1. Aim in one sentence

Build an edge-efficient large-language-model compression pipeline that replaces selected dense Transformer weight matrices with low-bond-dimension MPO (Matrix Product Operator) representations, applies quantization to the remaining layers, and recovers quality through calibration or fine-tuning.

The target is not simply the smallest checkpoint. The target is a useful Pareto point: substantially lower memory and storage, acceptable latency on the deployment device, and no unacceptable loss on the intended language and downstream tasks.

2. What the target paper contributes

The CompactifAI paper describes a pipeline with the following important ideas:

1. **Layer tensorization:** reshape selected LLM weight matrices and decompose

them into MPOs using sequential SVD truncation.

1. **Bond dimension as the compression control:** the MPO bond dimension

controls how much correlation and representational capacity is retained.

1. **Layer sensitivity profiling:** different Transformer blocks tolerate

tensorization differently. Early or particularly sensitive layers should be protected or assigned a larger bond dimension.

1. **Healing/retraining:** local truncation changes the model, so the

tensorized model is retrained to recover quality.

1. **Hybrid compression:** tensorize eligible layers and quantize the

remaining layers, rather than forcing one method on the complete model.

1. **System-level evaluation:** report memory, parameter reduction, training

cost, speed, and benchmark quality together.

The paper reports that combining tensor-network compression with quantization can substantially reduce memory for LLaMA-2-class models. Those numbers are paper results, not guarantees for this repository; they must be reproduced with the same model, data, hardware, and evaluation protocol before being claimed here.

3. Current repository position

This repository already contains a useful first implementation:

- `ResearchMPOLinear` performs explicit TT-SVD with a maximum `bond_dim`.
- `ResearchMPOCompressor` handles `nn.Linear` and GPT-2-style `Conv1D`

modules.

- `MPOCompressor` provides a legacy rank-ratio baseline.
- `MixedCompressionCompressor` combines MPO with dynamic int8 quantization.
- `CompressionPolicy` can protect selected module paths.
- Bundle manifests record compression settings.
- The accuracy evaluator compares original and compressed next-token behavior.

This is a **reproduction scaffold**, not yet a full CompactifAI implementation. The largest missing pieces are sensitivity-driven rank selection, a healing/training loop, robust benchmark reporting, and device-level performance validation.

4. Gap analysis

Target capability	Current status	Work required
MPO/TT-SVD decomposition	Present	Validate reconstruction error and shape conventions on every supported model family
Explicit bond dimension	Present in research path	Add per-layer or per-block budgets
Layer protection	Present through policy patterns	Replace manual defaults with measured sensitivity profiles
Hybrid MPO + quantization	Present as a baseline	Compare quantization backends and ensure the final runtime supports every layer type
Layer sensitivity profiling	Missing	Sweep layers/blocks and record quality change per parameter or memory saved
Healing/retraining	Missing or external to the compression call	Add calibration and optional QLoRA/PEFT healing
LLaMA-family support	Not established by the current scaffold	Add model loading, module mapping, tokenizer, and checkpoint compatibility tests
Paper-style evaluation	Partial	Add perplexity, downstream tasks, memory, latency, throughput, and energy where available
Reproducibility	Partial	Record revisions, seeds, data, hardware, software, ranks, skipped layers, and commands
Edge deployment	Bundle/export exists	Benchmark on the actual CPU/mobile/accelerator target and optimize contraction kernels

5. Work required to achieve the aim

Phase 0 — Freeze a reproducible baseline

Choose one supported, legally accessible model and one evaluation corpus before changing the compressor. A small GPT-2 or DistilGPT-2 model is appropriate for fast development; LLaMA-2 7B or another compatible model should be used for a paper-scale experiment only when the hardware and model license permit it.

Record:

- model identifier and exact revision;
- tokenizer and context length;
- Python, PyTorch, Transformers, and quantization-library versions;

- CPU/GPU model and available memory;
- random seeds;
- calibration and evaluation datasets;
- batch size and sequence length;
- baseline parameter count, serialized size, peak memory, latency, throughput,

and perplexity.

Exit criterion: the uncompressed model produces a saved baseline report and the run can be repeated from a documented command.

Phase 1 — Validate the MPO implementation

Before compressing an entire LLM, test `ResearchMPOLinear` independently:

1. Generate deterministic random matrices with dimensions representative of

attention and MLP projections.

1. Decompose them at several bond dimensions.
2. Reconstruct them and measure relative Frobenius error.
3. Compare the MPO forward pass with the original matrix multiplication.
4. Test rectangular matrices and GPT-2 `Conv1D` orientation.
5. Test serialization and reload.
6. Verify that `bond_dim` is an actual maximum internal bond, not an indirect

rank ratio.

Exit criterion: reconstruction error decreases monotonically or near-monotonically as bond dimension increases, full-rank settings preserve the original output within numerical tolerance, and save/load gives the same output.

Phase 2 — Map model modules and establish safe defaults

For each model family, enumerate all candidate modules and classify them:

- token embeddings and output head;
- attention query, key, value, and output projections;
- MLP up, gate, and down projections;
- normalization and other non-linear or unsupported modules.

Start conservatively. Protect embeddings, the language-model head, the first Transformer block, and any module that shows a large quality drop. Compress only modules with a valid matrix orientation and supported dimensions.

Exit criterion: a dry-run report lists every compressed and skipped module, the reason for each decision, and the expected parameter count before the actual weights are replaced.

Phase 3 — Implement layer sensitivity profiling

The target paper's most important practical lesson is that layers are not equally compressible. Build a profiler that:

1. compresses one layer or one block at a time;
2. evaluates a fixed calibration set;
3. measures loss/perplexity change and reconstruction error;

4. records the layer's original parameter count and estimated compressed size;
5. optionally repeats at multiple bond dimensions;
6. emits a machine-readable profile and a human-readable table.

Use the profile to produce a policy under a memory budget. A first policy can rank layers by quality loss per parameter saved. A better policy can solve a knapsack-like allocation problem over layer bond dimensions.

Exit criterion: the same memory budget produces a better quality result with the sensitivity-aware policy than with one global bond dimension.

Phase 4 — Add healing

One-shot truncation is only an initialization. Add two healing modes:

Calibration-only healing

- collect representative activations without labels;
- fit scales/statistics required by the quantized layers;
- optionally optimize only lightweight correction parameters;
- keep the base model frozen.

This mode should be fast and cheap enough for routine experiments.

PEFT/QLoRA healing

- train adapters or selected MPO cores on a small, documented corpus;
- use the same tokenizer and task distribution as evaluation;
- support mixed precision and gradient checkpointing;
- save the healing configuration and trainable parameter count;
- compare quality recovery against training cost.

Exit criterion: healing recovers a measurable fraction of the quality lost by compression without silently changing the evaluation data or model configuration.

Phase 5 — Build the hybrid compression path

Evaluate at least these configurations:

1. original model;
2. quantization only;
3. MPO only;
4. MPO plus quantization;
5. MPO plus healing;
6. MPO plus quantization plus healing.

For each configuration, record:

- parameter count and parameter ratio;
- serialized file size;
- peak resident and accelerator memory;
- cold and warm latency;
- throughput at the chosen batch and sequence sizes;
- perplexity or next-token loss;
- downstream task scores;
- healing time and trainable parameter count.

Do not infer runtime speed from parameter count. MPO contraction overhead can make a smaller model slower on a runtime without optimized tensor-network kernels.

Exit criterion: at least one hybrid configuration improves the target memory/quality trade-off over quantization-only on the selected hardware.

Phase 6 — Validate edge deployment

Select the actual deployment target: laptop CPU, Raspberry Pi, Jetson, mobile CPU/GPU, or another edge device. Then:

- export the compressed bundle;
- verify it can be loaded without the training environment;
- benchmark end-to-end generation, not only a layer microbenchmark;
- measure peak memory and sustained throughput;
- test multiple context lengths;
- check numerical stability and failure behavior;
- document unsupported operations and fallback paths.

Exit criterion: a fresh environment on the target device loads the bundle and produces reproducible output within the agreed memory, latency, and quality budgets.

6. Recommended experiment matrix

Experiment	Settings	Primary question
MPO reconstruction	`mpo_sites` 2–4; several `bond_dim` values	Does the implementation preserve the expected error/rank trade-off?
Global-rank baseline	One bond dimension for all eligible layers	How far can a simple policy go?
Sensitivity policy	Per-layer protection and bond dimensions	Which layers dominate quality loss?
Quantization baseline	int8 and, where supported, int4	What does quantization alone achieve?
Healing ablation	None, calibration, QLoRA/PEFT	How much quality can be recovered per unit of training?
Hardware benchmark	CPU, CUDA, and target edge device	Does lower storage translate into lower end-to-end cost?
Context-length sweep	Short, medium, and long prompts	Does compression degrade as sequence length grows?

Every row should use the same model revision, tokenizer, data split, seed, evaluation code, and reporting schema.

7. Definition of success

Set numeric thresholds before running the final experiment. A reasonable initial project target is:

- at least **50% lower serialized model size** than the chosen baseline;
- at least **30% lower peak memory** during inference;
- no more than **1 percentage point** loss on the selected language-quality

metric after healing;

- no unacceptable regression on the selected downstream tasks;
- no increase in warm end-to-end latency on the target device, unless the

explicit aim is storage-only compression;

- deterministic bundle loading and reproducible evaluation.

These are proposed engineering gates, not claims about the target paper's results. Adjust them for the selected model, hardware, and application.

8. Deliverables to create

1. `sensitivity_profile.json` and a rendered sensitivity table.
2. A baseline/compression benchmark JSON containing all metrics above.
3. A documented command that reproduces each experiment.
4. A healed compressed checkpoint and its manifest.
5. An edge-device benchmark log.
6. A final comparison table against the uncompressed and quantization-only

baselines.

1. A limitations section covering model license, dataset scope, unsupported

layers, and hardware-specific behavior.

9. Risks and mitigations

Risk	Mitigation
MPO shape factorization is unfavorable	Try alternate site counts/factorizations and report the chosen shape
Early layers or the output head are highly sensitive	Protect them or allocate larger bond dimensions
Compression saves storage but hurts latency	Benchmark real inference and add optimized contraction kernels
Healing overfits the calibration data	Use a held-out evaluation split and multiple task types
Quantized and MPO modules do not serialize together	Add round-trip bundle tests before large experiments
Results cannot be compared with the paper	Record exact model, data, hardware, ranks, and evaluation protocol
Large-model experiments exceed available hardware	Develop on a small model, then run a clearly labeled scale-up experiment

10. Immediate next actions

1. Add unit tests for MPO reconstruction, forward equivalence, rectangular

matrices, and serialization.

1. Add a dry-run module inventory to the compression CLI.
2. Implement a one-layer sensitivity profiler and JSON output.
3. Add perplexity/next-token-loss reporting alongside parameter and size

metrics.

1. Implement a small calibration/healing experiment before attempting a

multi-billion-parameter model.

1. Run the six-configuration matrix in Phase 5 on one fixed model and target

device.

1. Publish the resulting benchmark files with the compressed bundle so the

reported aim is auditable.

Conclusion

To achieve the aim represented by the supplied CompactifAI paper, the project must move from “an MPO decomposition exists” to “a measured, sensitivity-aware, healed, and deployable compression workflow exists.” The current code already provides the decomposition and hybrid-compression foundation. The critical remaining work is profiling, recovery training, reproducible evaluation, and real-device benchmarking. Those steps should be completed in that order so that a smaller model is not mistaken for a better edge model.